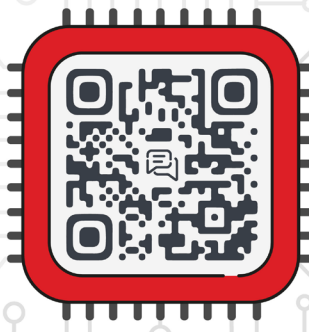




4



15



2250 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

AVL Trees



د. مدحت الصوص

RB Informatics; 3/5/2026

محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

تشمل هذه المحاضرة الأفكار التالية:

1. التعريف بالأشجار المتوازنة (Balanced Trees).
2. دراسة شجرة AVL.
3. التعرف على مفهوم الدورانات (Rotation) وأنواعها.
4. دراسة كيفية استخدام الدورانات في:
  - عملية إضافة العناصر.
  - عملية حذف العناصر.

🔑 مقدمة:

تحدثنا في المحاضرات السابقة، أن تعقيد البحث في أشجار البحث الثنائية هو  $O(h)$ ، حيث  $h$  هو ارتفاع الشجرة. لكن في حال أصبحت الشجرة غير متوازنة نتيجة عمليات الإضافة أو الحذف، قد يصل ارتفاعها إلى  $n$ ، عندها تفقد الشجرة كفاءتها، وتصبح مشابهة لبنية خطية. لذلك نلجأ إلى الأشجار المتوازنة للحفاظ على كفاءة العمليات.

الشجرة المتوازنة (Balanced Tree):

هي شجرة ثنائية يتم فيها الحفاظ على توازن الارتفاع، بحيث لا يزداد ارتفاع أحد الفرعين (الأيسر أو الأيمن) بشكل كبير على حساب الفرع الآخر.

أشهر أنواع الأشجار المتوازنة هي:

**AVL Tree:** تحافظ على الفرق بين ارتفاع الفرعين (الأيسر والأيمن) ولكي تُعتبر الشجرة شجرة AVL، يجب أن تحقق الشروط التالية:

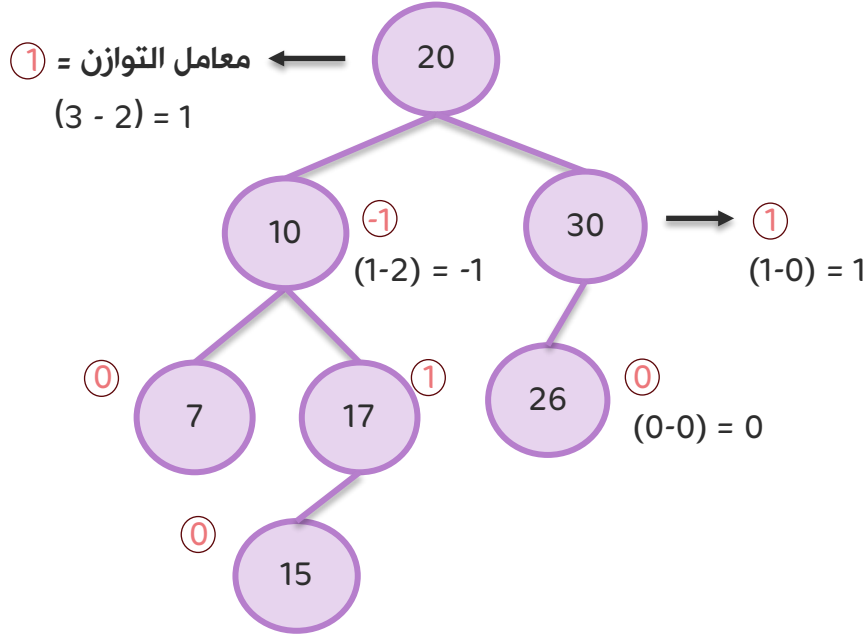
1. أن تكون شجرة بحث ثنائية (BST).
2. لكل عقدة في الشجرة، يجب أن تكون قيمة معامل التوازن حصراً تنتمي للمجموعة  $\{-1, 0, 1\}$ .

## معامل التوازن Balance Factor

$$\text{Balance Factor} = h(\text{left}) - h(\text{right})$$

- يعرف على أنه:
- أي إنه الفرق بين:
- ارتفاع الفرع الأيسر، وارتفاع الفرع الأيمن.

مثال:



■ ما الفائدة من قيد معامل التوازن:

يضمن عدم وجود فرق كبير في الارتفاع بين الفرعين الأيسر والأيمن، وبالتالي يتم التحكم بارتفاع الشجرة  $h$  ليبقى ضمن حدود لوغاريتمية.

نتيجة رياضية:

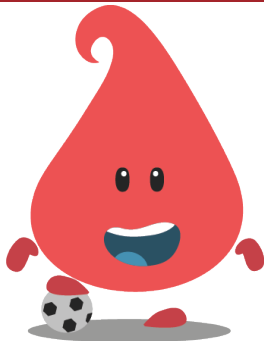
عند أفضل حالة

عند أسوأ حالة

$$\log(n+1) \leq h + 1 \leq 1.44 \log(n+2)$$

نلاحظ ان في أسوأ الحالات لعمليات (الإضافة – البحث – الحذف). يكون التعقيد  $1.44 \log(n+2)$  بدلاً من  $O(n)$  وهذا يدل على ان المشكلة قد تم حلّها.

■ أي أن التعقيد في ارتفاع شجرة AVL في أسوأ الحالات لا يتجاوز 44% من الارتفاع في أفضل الحالات (وهذا شيء عظيم جداً في عالم البرمجة).



ليس الخطأ أن تُخطئ،  
بل الخطأ أن تستمر بارتكاب الأخطاء

## كيف نحافظ على قيد معامل التوازن:

للمحافظة على القيد بكلفة منخفضة نستخدم الدورانات (Rotations).

### تعريف الدورانات:

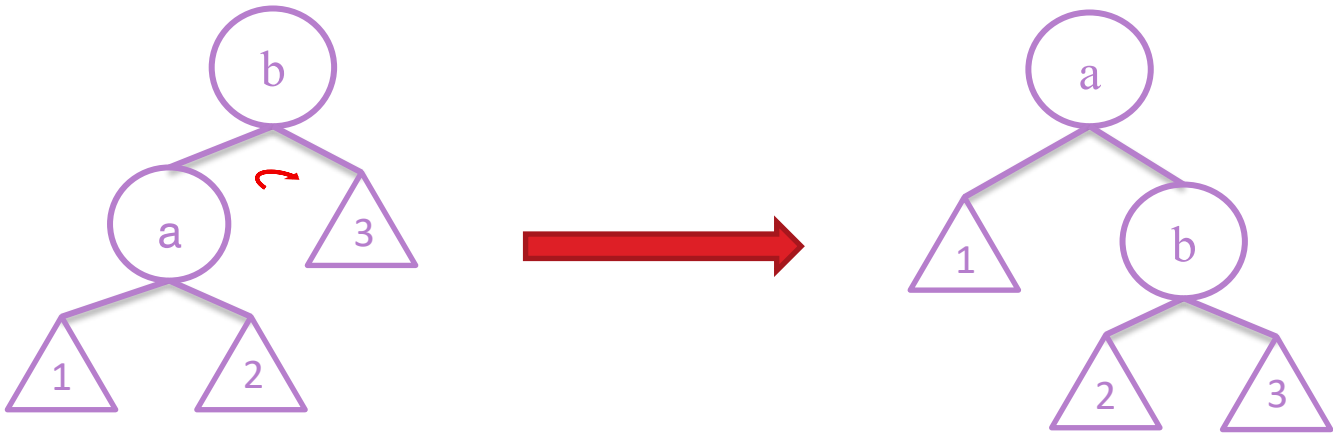
- هي عمليات تجرى على الشجرة الثنائية بهدف إعادة تحقيق التوازن بعد حدوث خلل نتيجة عمليات الإضافة أو الحذف.
- الفكرة ببساطة:
- نغير مواقع بعض العقد بشكل مدروس دون كسر خاصية ترتيب البحث (BST).

### أنواع الدورانات الأساسية:

#### 1. دوران يميني (Right Rotation):

- نستخدمه لما يكون في ميلان نحو اليسار.
- الفكرة ببساطة:
- العقدة اليسارية تطلع إلى الأعلى (مع الفرع اليساري لها).
- العقدة الأصلية تنزل إلى الأسفل (مع كل شيء أكبر منها).
- الابن اليميني للعقدة اليسارية ينتقل ليصبح ابن يساري للعقدة الأصلية.
- يمكن تصور الأمر على أنه (نثبت العقدة اليسارية كمركز دوران وندير العقدة باتجاه اليمين).

**مثال:**



- العقدة 'a' تطلع للأعلى هي وفرعها اليساري.
- العقدة 'b' تنزل للأسفل مع العناصر الأكبر منها.
- الوصلة المتبقية (الابن الأيمن لـ 'a') ينتقل ليصبح ابن أيسر للعقدة 'b'.



لم ننتهي بعد ^\_^

## 2. دوران يساري (Left Rotation):

- عكس السابق تماماً، نستخدمه لما يكون في ميلان نحو اليمين.
- الفكرة ببساطة:
- العقدة اليمينية تطلع إلى الأعلى (مع الفرع اليميني لها).
- العقدة الأصلية تنزل إلى الأسفل (مع كل شي أصغر منها).
- الابن اليساري للعقدة اليمينية ينتقل ليصبح ابن يميني للعقدة الأصلية.
- يمكن تصور الأمر على أنه (نثبت العقدة اليمينية كمركز دوران وندور العقدة باتجاه اليسار).

مثال:

- العقدة b تطلع للأعلى هي وفرعها اليميني.
- العقدة a تنزل للأسفل مع العناصر الأصغر منها.
- الوصلة المتبقية (الابن الأيسر ل b) ينتقل ليصبح ابن يمين للعقدة a.

## 3. دوران يساري يميني (Left-Right Rotation):

- هو عبارة عن تطبيق دورانيين متتاليين، دوران يسار ثم دوران يمين.
- ويستخدم عندما يكون الخلل في الجهة اليسرى لكن داخل الفرع الأيمن من العقدة اليسارية.

الفكرة ببساطة:

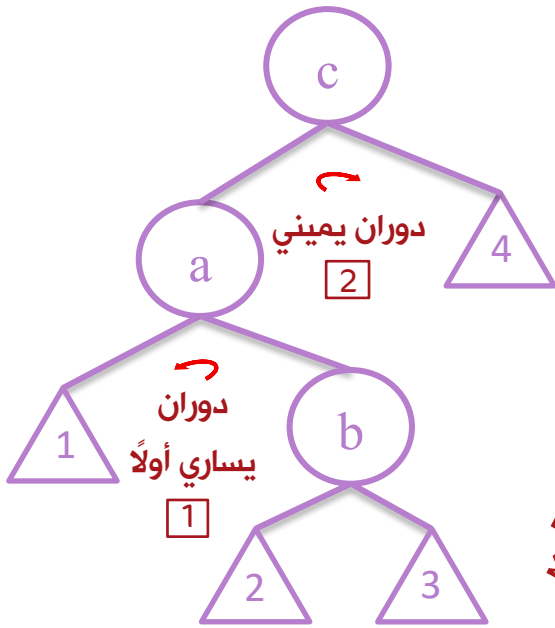
نبدأ من اليسار.

- نطبق الدوران اليساري على العقدة السفلية (الأبن).
- نطبق الدوران اليميني على العقدة العلوية (الأب).



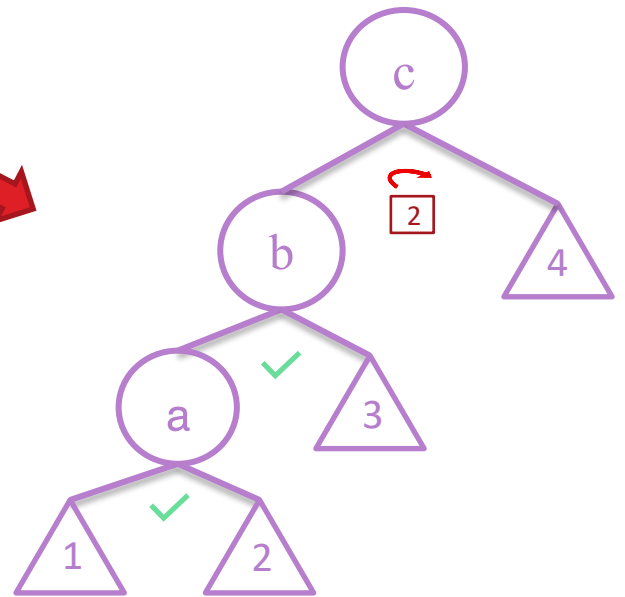
تذكر،

بعد سنة من الان ستكون مرفع المادة 😊

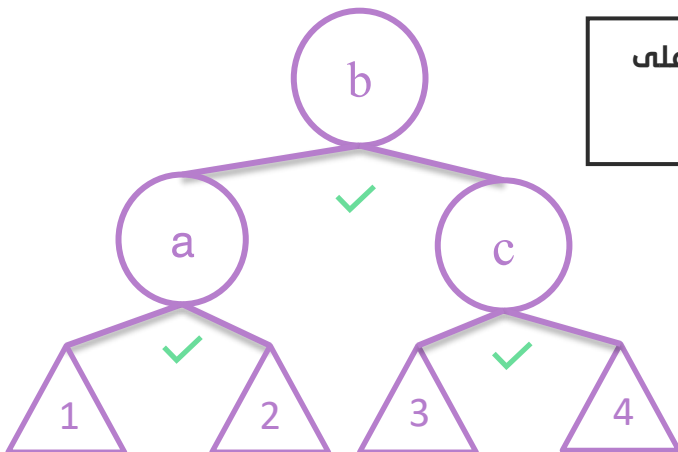


- نبدأ من الأسفل.
- نطبق الدوران اليساري (كما اخذناه سابقاً) على العقدة السفلية (الأبن) a.

نطبق دوران يساري أولاً



نطبق دوران يميني



- نطبق الدوران اليميني (كما اخذناه سابقاً) على العقدة العلوية (الأب) c.

تم حل المشكلة 😊

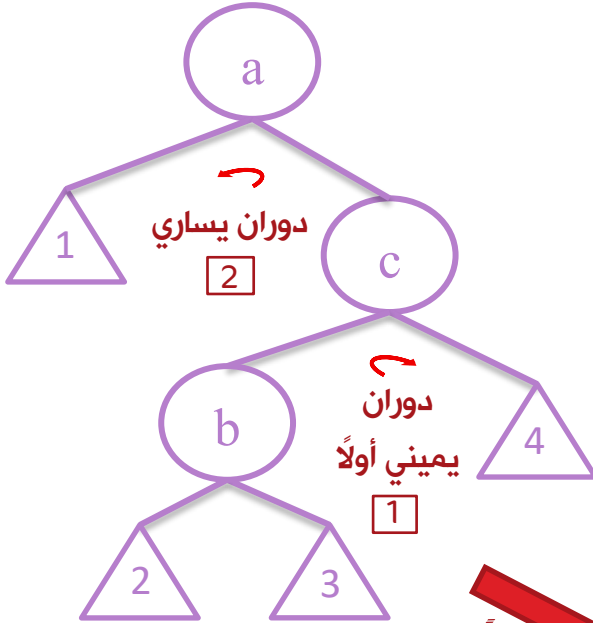
## 4. دوران يميني يساري (Right-Left Rotation):

هو أيضاً عبارة عن تطبيق دورانين متتاليين، دوران يمين ثم دوران يسار. ويستخدم عندما يكون الخلل في الجهة اليمنى لكن داخل الفرع الأيسر من العقدة اليمينية.

الفكرة ببساطة:

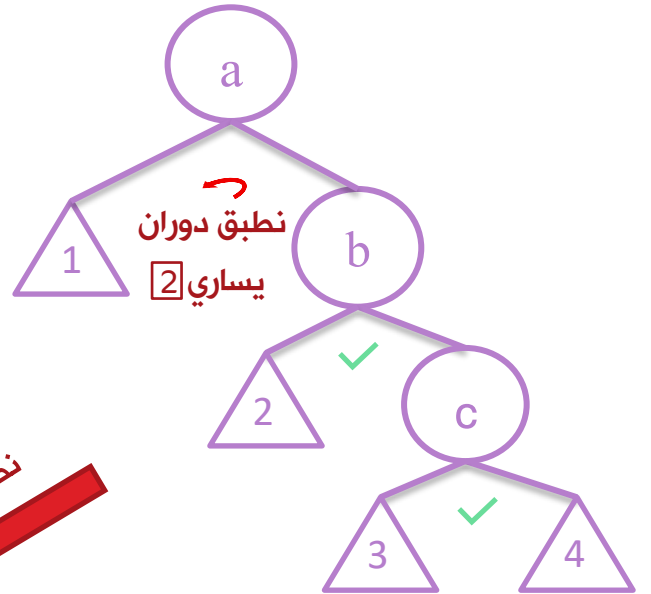
- نطبق الدوران اليميني على العقدة السفلية (الأبن).
- نطبق الدوران اليساري على العقدة العلوية (الأب).

**مثال:**

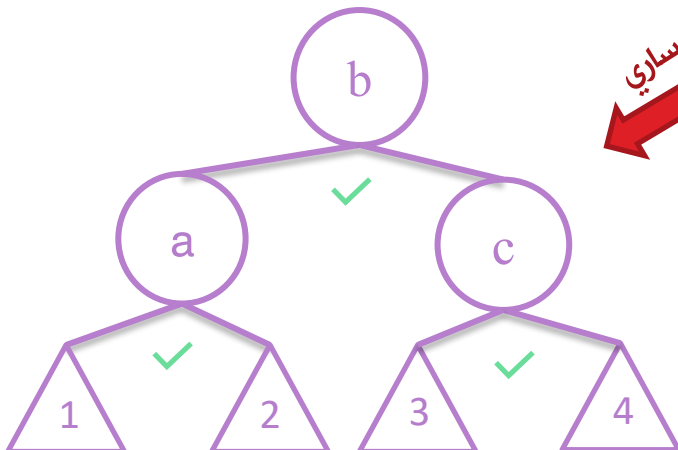


- نبدأ من الأسفل.
- نطبق الدوران اليميني (كما اخذناه سابقاً) على العقدة السفلية (الأبن) c.

نطبق دوران يميني أولاً



نطبق دوران يساري



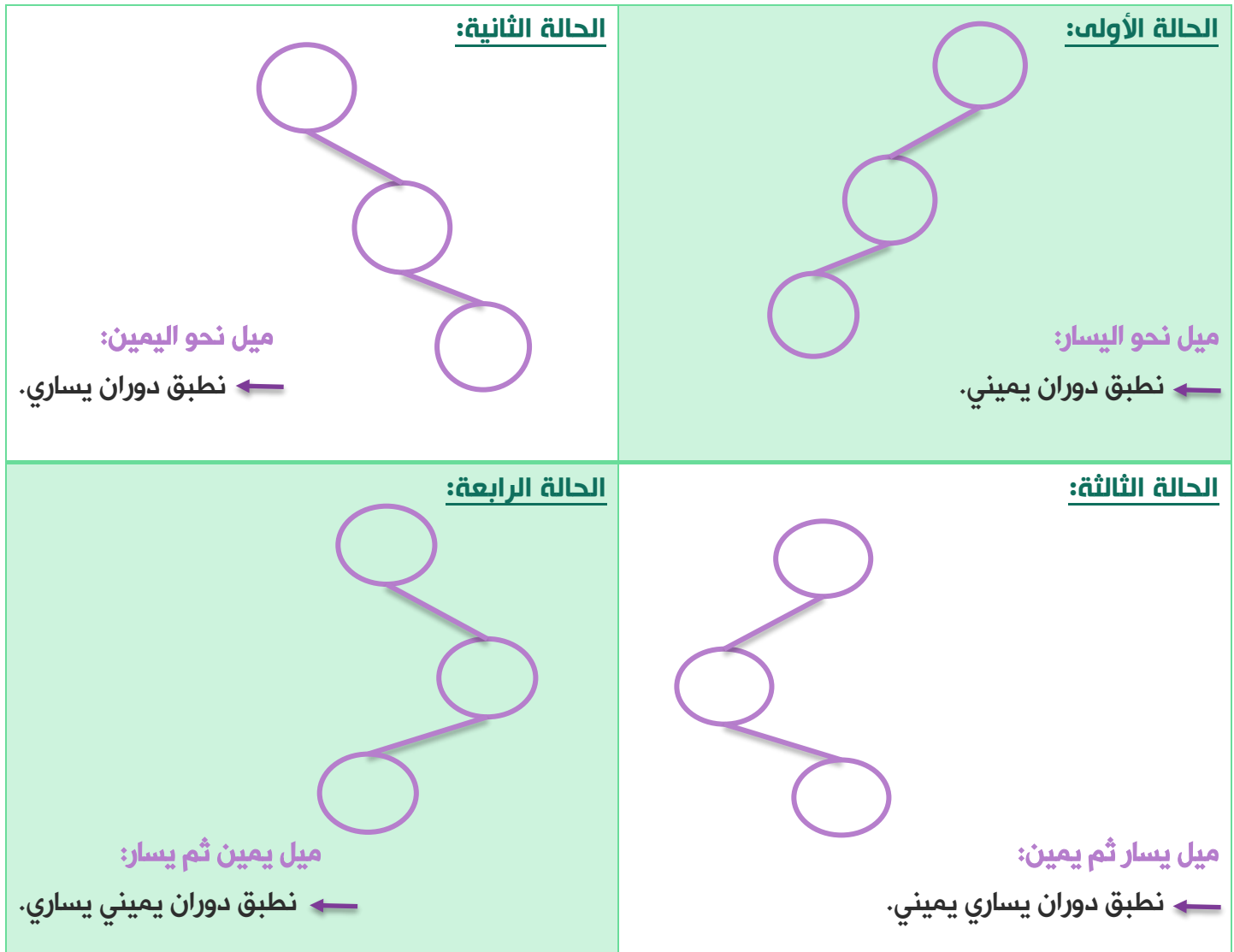
- نطبق الدوران اليساري (كما اخذناه سابقاً) على العقدة العلوية (الأب) a.



تم حل المشكلة.. ببساطة

## كيف نحدد نوع الدوران المناسب:

لتحديد نوع الدوران، نقوم بفحص العقدة التي حدث عندها خلل (عدم توازن).



طريقة أخرى لتحديد نوع الدوران المناسب:

عن طريق إشارة معامل التوازن.

| دوران يساري                                                | دوران يميني                                                |
|------------------------------------------------------------|------------------------------------------------------------|
| معامل التوازن لكل من عقدة الخلل وابنها في ال subtree سالب. | معامل التوازن لكل من عقدة الخلل وابنها في ال subtree موجب. |
| دوران يميني يساري                                          | دوران يساري يميني                                          |
| معامل التوازن لعقدة الخلل سالب بينما الأبْن موجب.          | معامل التوازن لعقدة الخلل موجب بينما الأبْن سالب.          |

ملاحظة 1: الدوران لا يغير ترتيب عناصر شجرة البحث الثنائية BST.

أي إن العناصر تبقى مرتبة كما هي، وإنما يتغير شكل الشجرة فقط لإعادة توزيع الارتفاع.

ملاحظة 2: نحن لا نقوم بإعادة بناء شجرة من جديد، وإنما نكتفي بتعديل بعض المؤشرات (الوصلات) بين العقد فقط.

## استخدام الدورانات:

### 1. عند الإضافة:

تتم عملية الإضافة في شجرة AVL على مرحلتين:

1. تنفيذ الإضافة: تطبيق نفس خطوات الإضافة في شجرة البحث الثنائية (BST).  
(يتم إدراج العنصر الجديد كورقة في مكانه المناسب).
2. التحقق من التوازن وإصلاح الخلل.

### كيفية إصلاح الخلل:

- نبدأ بالفحص من العقدة الجديدة باتجاه الجذر (بحساب معامل التوازن لكل عقدة على المسار والتحقق من انتمائه للمجموعة  $\{-1, 0, 1\}$ ).
- عند العثور على أول عقدة غير متوازنة، يتم تطبيق الدوران المناسب (اعتماداً على الحالات السابقة التي تعلمناها).

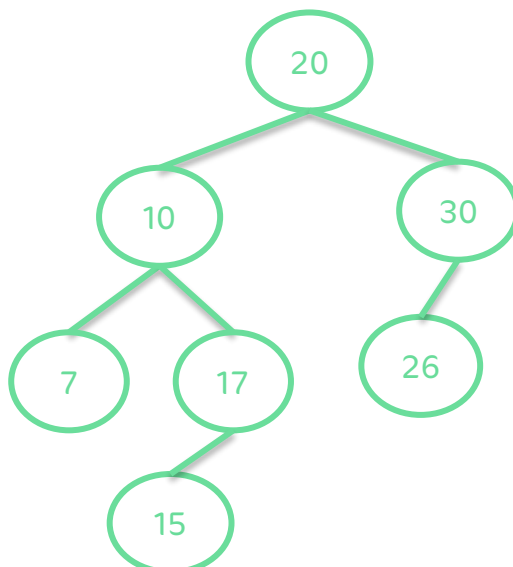
ملاحظة 1: في حال عدم الاختلال فلن نقوم بأي تعديل، وسوف تبقى الشجرة كما هي لأن شرط التوازن مازال محققاً.

ملاحظة 2: نبحث عن أول عقدة يظهر فيها خلل في التوازن أثناء الصعود باتجاه الجذر، عند إيجادها نقوم بإصلاحها مباشرة باستخدام الدوران المناسب ثم نتوقف لأن الشجرة تكون قد عادت إلى حالة التوازن (وهذا في حال حدوث الخلل فقط عند عملية الإضافة).

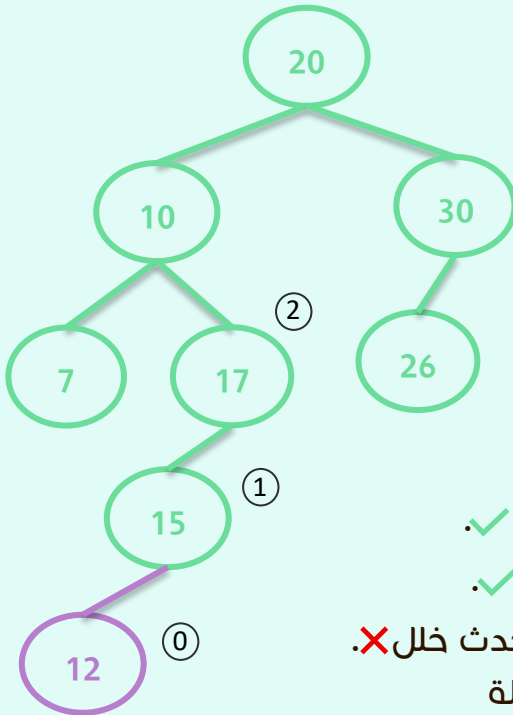
### مثال:

لتكن لدينا الشجرة التالية:

- نريد إضافة العدد (12).



## الحل:



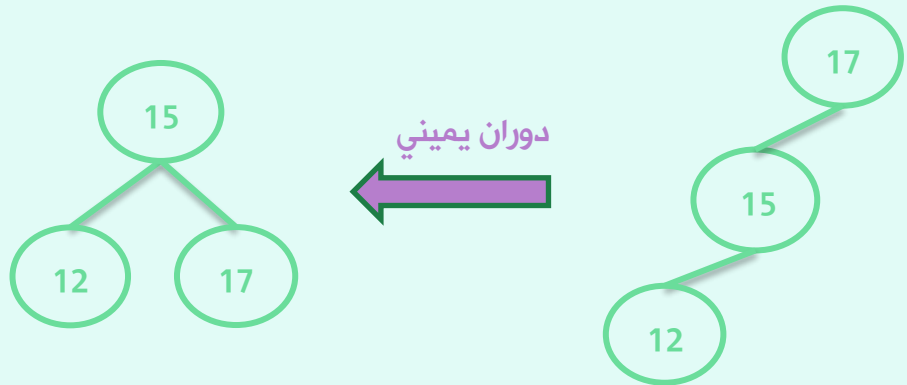
1. نبحث عن المكان المناسب له ونضيفه.

2. نبدأ من العقدة المضافة باتجاه الجذر، ونقوم بفحص معامل التوازن لكل عقدة تمر على المسار. نبدأ:

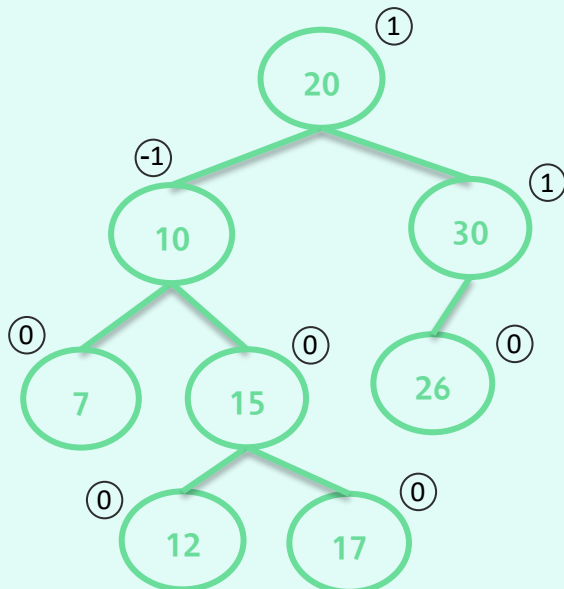
معامل التوازن عند العقدة 12 يساوي 0 ← ضمن الشرط ✓.

معامل التوازن عند العقدة 15 يساوي 1 ← ضمن الشرط ✓.

نلاحظ عند العقدة 17 معامل التوازن يساوي 2 ← هنا يحدث خلل ✗. وبحسب شكل المسار للعقدة التي حدث فيها الخلل فإن الحالة ميل نحو اليسار ← لذلك نستخدم دوراناً يمينياً للإصلاح.

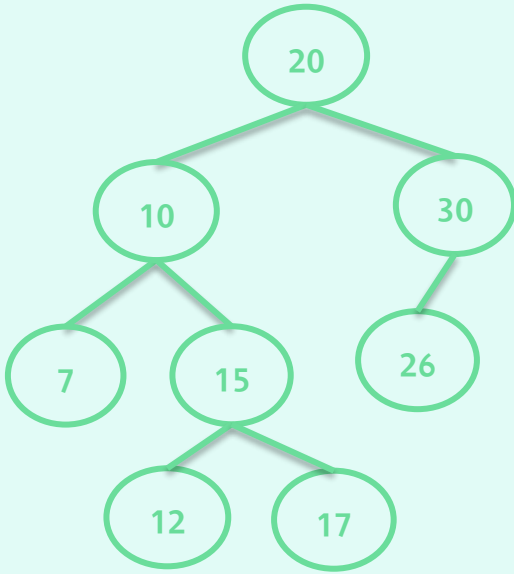


ومنه الشجرة الجديدة بعد الإصلاح:



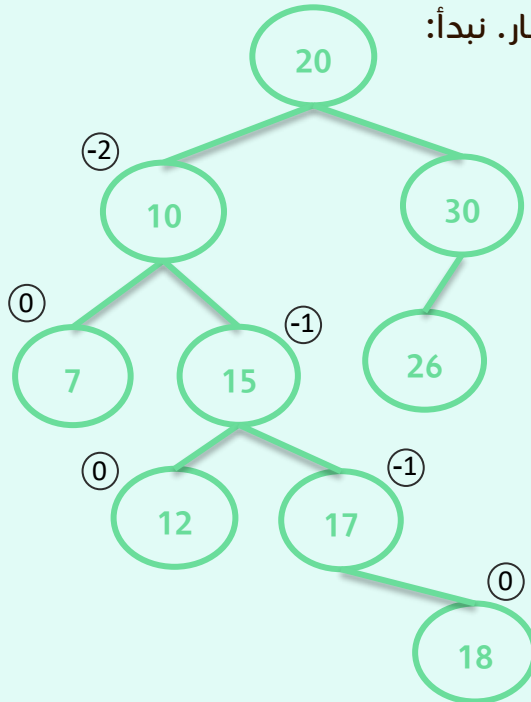
(يكفي حل أول عقدة حصل فيها الخلل)  
تمام..

## ■ إضافة العقدة (18):



1. نبحث عن المكان المناسب لها ونضيفها.

2. نبدأ من العقدة المضافة باتجاه الجذر، ونقوم بفحص معامل التوازن لكل عقدة تمر على هذا المسار. نبدأ:

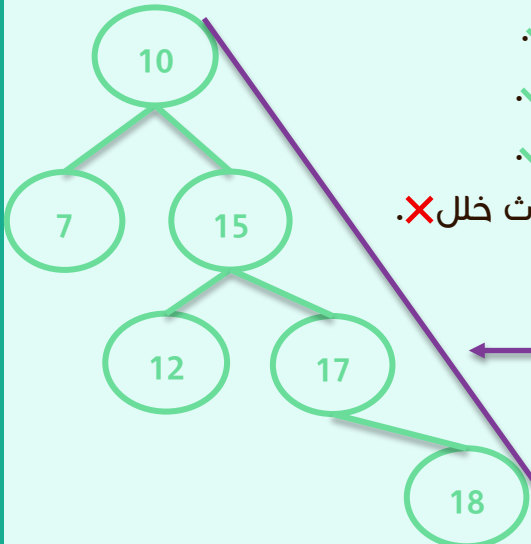


■ معامل التوازن عند العقدة 18 يساوي 0 ← ضمن الشرط ✓.

■ معامل التوازن عند العقدة 17 يساوي -1 ← ضمن الشرط ✓.

■ معامل التوازن عند العقدة 15 يساوي -1 ← ضمن الشرط ✓.

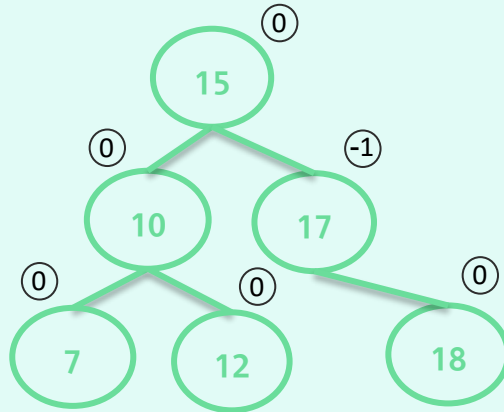
■ نلاحظ عند العقدة 10 معامل التوازن يساوي -2 ← هنا يحدث خلل ✗. وبحسب شكل المسار للعقدة التي حدث فيها الخلل.



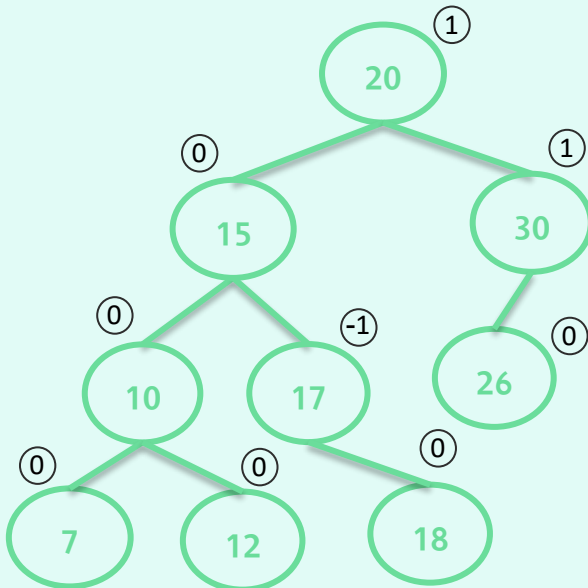
فإن الحالة:

ميل نحو اليمين ← لذلك نستخدم دوراناً يسارياً لإصلاح الخلل.

نطبق دوران يساري على العقدة (10).  
لتصبح:



ومنه الشجرة الجديدة بعد الإصلاح:



## 2. عند الحذف:

1. تتم عملية الحذف أولاً بنفس طريقة شجرة البحث الثنائية (BST) وذلك وفق الحالات التالية:

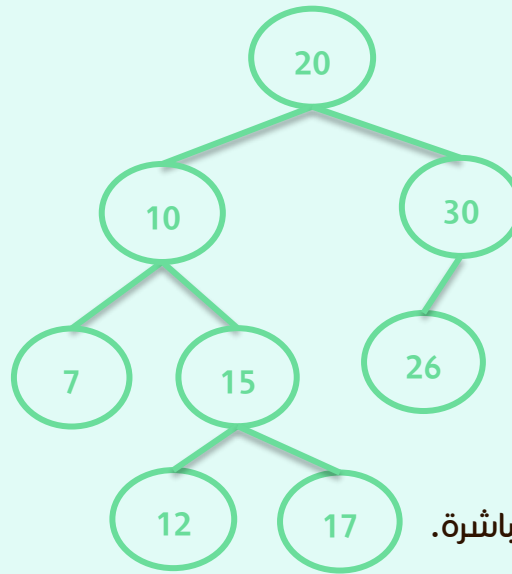
- حذف عقدة لا تمتلك أبناء (يتم حذفها مباشرة).
- حذف عقدة لها ابن واحد (يتم ربط الاب مباشرة بالابن).
- حذف عقدة لها ابنان (يتم استبدالها بالعنصر التالي ثم حذف هذا العنصر من مكانه الأصلي).

2. إعادة التوازن بعد الحذف:

- يتم فحص الشجرة، نبدأ من الأسفل إلى جهة الجذر، وكلما وجدنا خلل نقوم بإصلاحه ومن الممكن أن نقوم بأكثر من إصلاح عند حذف عقدة واحدة.

**مثال:** ليكن لدينا الشجرة التالية:

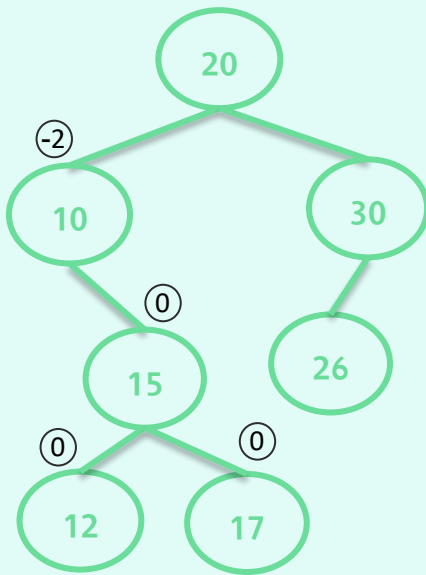
■ نريد حذف العدد (7).



■ **الحل:**

1. لنحذف العدد (7):

■ عقدة ليس لها أبناء نحذفها مباشرة.

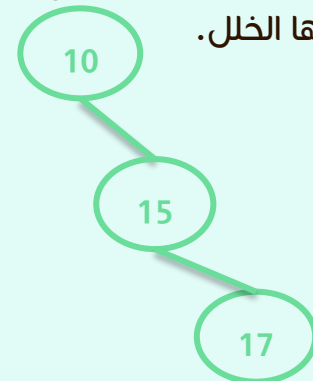


2. نفحص العقدة.

عند العقدة 10 معامل التوازن يساوي -2 ← اختل شرط التوازن ✗.

■ للإصلاح:

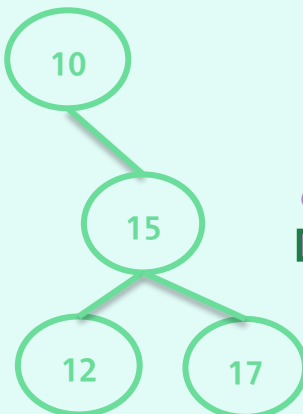
نأخذ ال subtree ذات الارتفاع الأطول للعقدة 10 التي حصل عندها الخل.



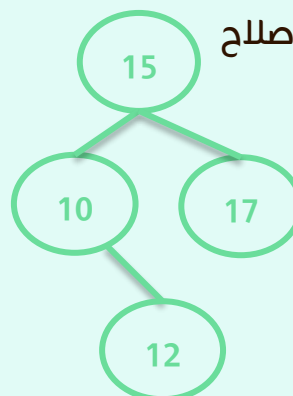
ال subtree ذات الارتفاع الأطول

■ نلاحظ من شكل المسار: الحالة من الشكل ميل نحو اليمين.

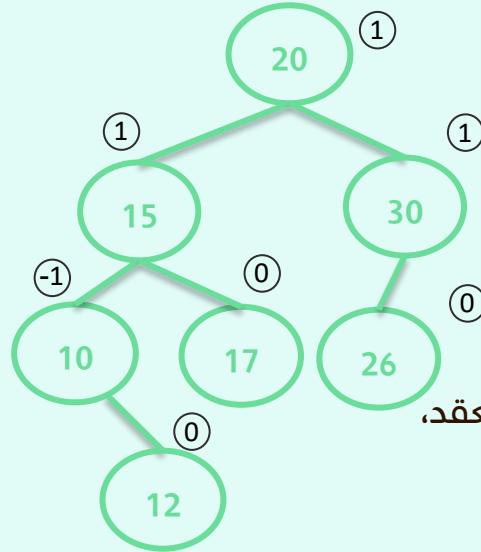
لذلك نستخدم دوران يساري للإصلاح



دوران يساري



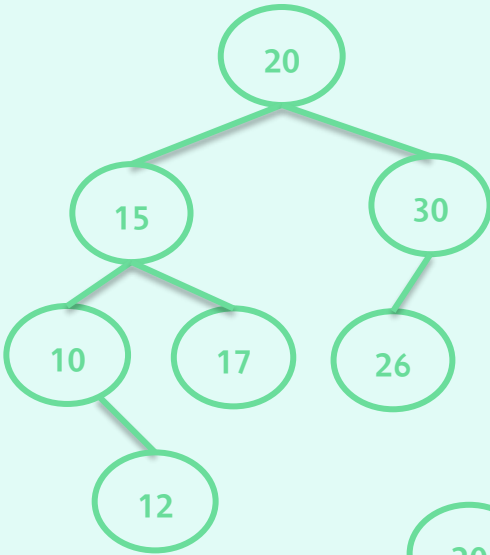
■ ومنه الشجرة الجديدة بعد الإصلاح:



■ نكمل فحص العقد

■ نلاحظ أن معاملات التوازن في كل العقد، ضمن المجال المحدد.

■ حذف العدد (15).



1. لحذف العدد (15):

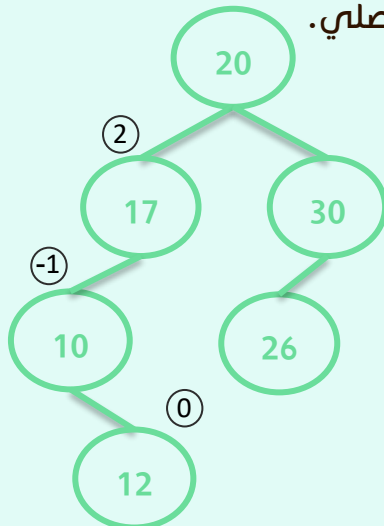
■ عقدة لها ابنين.

1- نبحث عن العنصر التالي.

2- نستبدل العقدة 15 بالعنصر التالي.

3- نحذف العنصر التالي من مكانه الأصلي.

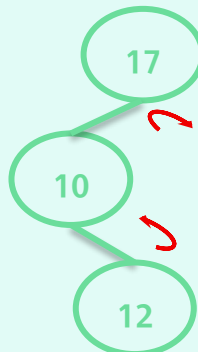
← العنصر التالي هو 17 نستبدله ونحذفه من مكانه الأصلي.



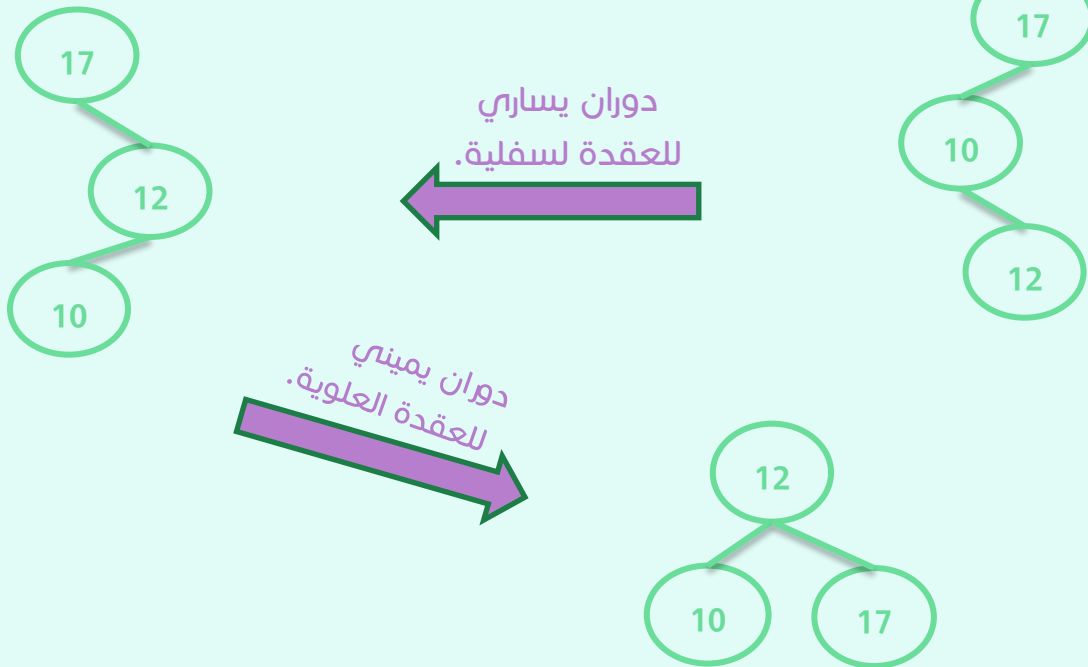
2. نفحص العقد:

معامل التوازن للعقدة 17 يساوي 2،

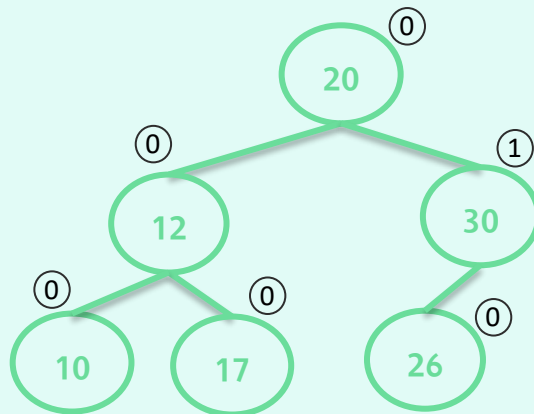
← اختل شرط التوازن. ✗



■ نلاحظ من الشكل السابق:  
يحتاج دوران يساري يميني.



ومنه الشجرة الجديدة بعد الإصلاح:



نكمل فحص العقد.

■ نلاحظ أن معاملات التوازن في كل العقد ضمن المجال المحدد.

#### تعقيد الحذف:

■ تعقيد الحذف  $O(h)$  من رتبة الارتفاع.

■ تعقيد الإصلاح  $O(h)$  من رتبة الارتفاع.

$$O(H+H) = O(2H) = O(H)$$

وبما إن الشجرة AVL متوازنة دائماً، فإن التعقيد:

$$O(H) = O(\log_2 n)$$

## الفرق بين الإضافة والحذف:

| الحذف                                                                                                         | الإضافة                                                                                |                                            |
|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|--------------------------------------------|
| نبدأ من العقدة التي حصل عندها الحذف. نستمر بإصلاح كل خلل يظهر حتى الوصول إلى الجذر، (لا نتوقف عند أول إصلاح). | نبدأ من العقدة المضافة أول عقدة يظهر فيها خلل يتم إصلاحها مباشرة، ثم نتوقف عن الإصلاح. | الإصلاح                                    |
| نأخذ العقدة التي حصل فيها الخلل مع الفرع (الابن) الذي يحتوي ارتفاع أكبر (نركز على الفرع الأكبر).              | المسار: العقدة التي حصل فيها الخلل+ابنها + حفيدها.                                     | اختيار مسار الإصلاح (لاختيار نوع الدوران). |

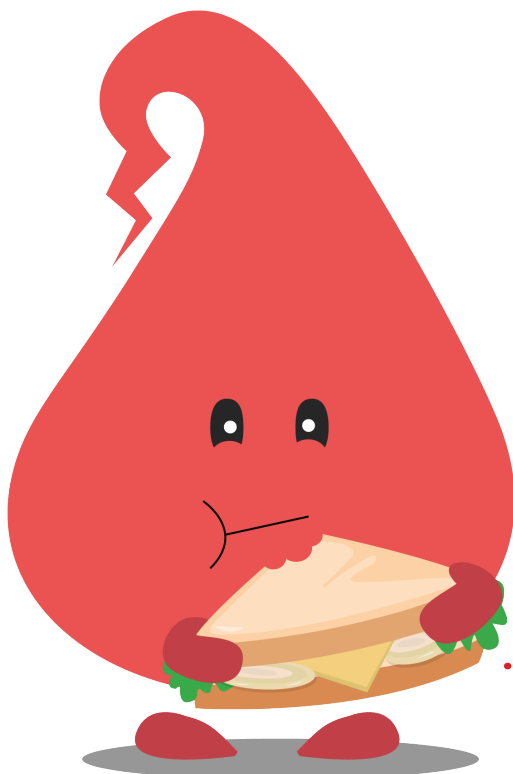
## ملخص لكل الأفكار:

ال AVL: هي شجرة بحث ثنائية نطبق عليها قيد لتقليل ارتفاع الشجرة بأقصى حد ممكن،

تعقيد الحذف والإضافة فيها يكون من رتبة الارتفاع:  $O(H) = O(\log_2 n)$ .

في الإضافة نطرح مرة واحدة باستخدام نوع مناسب من الدورانات،

في الحذف من الممكن أن نضطر للإصلاح أكثر من مرة طالما يوجد خلل في مسار البحث، ومن الممكن نستخدم عدة أنواع من الدورانات (حسب شكل ال subtree التي وقع فيها الخطأ).



قال حكيم ذات مرة:

عندما تواجهك مشكلة في الخوارزمية،

تذكر ان الحل في الكود وليس في كيس الشيبس..